C/C++-Kurs für Kinder ab der 5. Klasse

Jürgen “George” Sawinski

6. August 2026

Zusammenfassung

Dieser Kurs ist gedacht für Kinder ab Ende der Grundschule (ca. 5. Klasse), die Begeisterung für Mathematik und Computer mitbringen und den Drang haben, programmieren zu lernen.

Der erste Teil des Kurses beschäftigt sich damit, alle grundlegenden Kenntnisse zu erwerben, um eine Konsolen-basierte Regensimulation in C zu programmieren, was auch mathematische und physikalische Grundlagen beinhaltet.

Der zweite Teil des Kurses wird das Ziel haben, das Spiel Tetris® in C++ umzusetzen, wobei das Hauptaugenmerk dabei auf die Grundlagen Objekt-orientierter Programmierung gelegt wird.

Inhaltsverzeichnis

1 Etwas Mathematik 3
- 1.1 Das Binärsystem . . . 3
- 1.2 Binäre Operationen — wie Computer rechnen . . . 5
- 1.3 Logische Operationen . . . 6

2 C-Grundlagen 8
- 2.1 Hallo Welt! . . . 8
- 2.2 Zahlen . . . 10
- 2.3 Fibonacci . . . 11
- 2.4 Reihen . . . 13
- 2.5 Lauflicht . . . 14

A Software 15

B Leitfaden 16
- B.1 Tastenkombinationen . . . 16
  - B.1.1 Sonderzeichen . . . 16
  - B.1.2 Referenz . . . 17
- B.2 C . . . 18
  - B.2.1 Grundlagen . . . 18
  - B.2.2 Referenz . . . 25
- B.3 Lösungen zu den Übungen . . . 26

Vorwort

Dieser Kurs ist aus der gemeinsamen Arbeit des Autors mit seinen Söhnen entstanden. Die Idee, einen C/C++-Kurs für Kinder zu schreiben, kam von den Jungs selbst, die unbedingt programmieren lernen wollten um ihre eigenen Spiele zu entwickeln zu können.

Die Mitschrift ist in erster Linie für Kinder gedacht, die den Kurs mit einer Begleitperson zusammen durchlaufen und dient als Leitfaden sowie als Nachschlagewerk für die behandelten Themen.

Der Autor hat die Inhalte erarbeitet, erklärt und in LaTeX festgehalten. Bei der Strukturierung des Materials und der Erstellung einiger LaTeX-Makros wurde er durch KI-gestützte Werkzeuge (DeepSeek und GitHub Copilot) unterstützt.

DeepSeek lieferte Vorschläge zur Gliederung, zur didaktischen Aufbereitung und zu technischen Umsetzungen in LaTeX. Github Copilot wurde beim Schreiben von LaTeX-Code und bei der Erstellung von C/C++-Codebeispielen eingesetzt. Alle Vorschläge wurden vom Autor geprüft, angepasst und auf die Bedürfnisse seiner Kinder zugeschnitten.

– Jürgen "George" Sawinski, 6. August 2026

Lizenz und Haftungsausschluss

Alle Inhalte dieses Kurses wurden vom Autor selbst erstellt und sind, soweit nicht anders gekennzeichnet, frei von urheberrechtlich geschützten Materialien Dritter. Die Beispiele und Übungen sind so gestaltet, dass sie die Lernziele unterstützen und den Kindern helfen, die Konzepte zu verstehen und anzuwenden.

Dieses Werk wird unter der MIT-Lizenz veröffentlicht. Das bedeutet: Ihr dürft es kopieren, verändern, weitergeben und für eigene Zwecke nutzen — auch kommerziell. Die einzige Bedingung ist, dass der Autor (Jürgen "George" Sawinski) in der Quellenangabe genannt wird.

Haftungsausschluss: Dieser Kurs wurde mit größter Sorgfalt erstellt. Dennoch übernimmt der Autor keine Gewähr für die Richtigkeit, Vollständigkeit oder Aktualität der Inhalte. Die Nutzung der Inhalte erfolgt auf eigene Verantwortung. Der Autor haftet nicht für Schäden, die aus der Nutzung oder der Unfähigkeit zur Nutzung der Inhalte entstehen.

Projekt-Webseite: `https://github.com/jsawinski/twin-c-course`

1 Etwas Mathematik

1.1 Das Binärsystem

Computer rechnen im sogenannten Binärsystem, sie kennen nur 0 und 1. Um größere Zahlen darstellen zu können, werden mehrere dieser Nullen und Einsen, genannt "Bit" aneinandergereit. Am gebräuchlichsten dabei ist das "Byte" (siehe Abb. 1), das aus 8 Bit besteht.

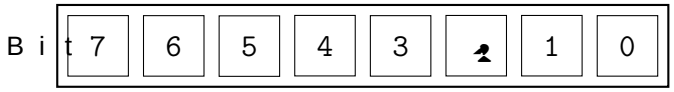


Abbildung 1: Ein Byte mit den Bit-Positionen 0 bis 7.

Jedem Bit wird nun ein Wert zugeordnet. Das erfolgt mathematisch gemäß:

$$2^b$$

wobei b die Nummer des Bits ist. Das ist eine sogenannte "Potenz", eine mathematische Operation die sagt, wie of eine Zahl mit sich selbst multipliziert wird. In diesem Fall wird die "2" b-mal mit sich selbst multipliziert, zum Beispiel:

$$\begin{aligned} 2^1 &= 2 && = 2 \\ 2^2 &= 2 \cdot 2 && = 4 \\ 2^3 &= 2 \cdot 2 \cdot 2 && = 8 \end{aligned}$$

Interessant ist nun die Frage, was 2^0 ist. Wenn ihr genau hinschaut, dann merkt ihr, dass, um in die nächste Zeile zu kommen, ihr das Ergebnis mit 2 multipliziern müsst. Umgekehrt genauso: um in die vorhergehende Zeile zu kommen, muss man durch 2 teilen. Wenn also 2^1 zwei ist und man, um zu 2^0 zu kommen durch zwei Teilen muss, dann ergibt sich

$$2^0 = 1\,.$$

Übrigens, jede Zahl hoch 0 hat als Ergebnis die 1 (wir kommen später darauf zurück).

Um auf das Byte zurückzukommen, heißt dass, dass den Bits jeweils die Werte 1 bis 128 zugeordnet sind (siehe Abb. 2).

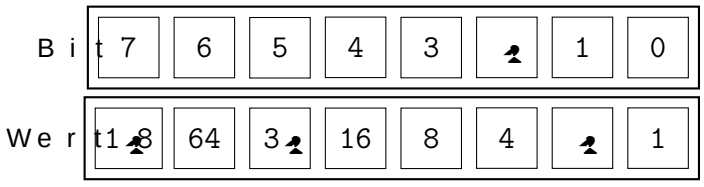


Abbildung 2: Ein Byte mit Bit-Positionen und den zugehörigen Werten (2^b).

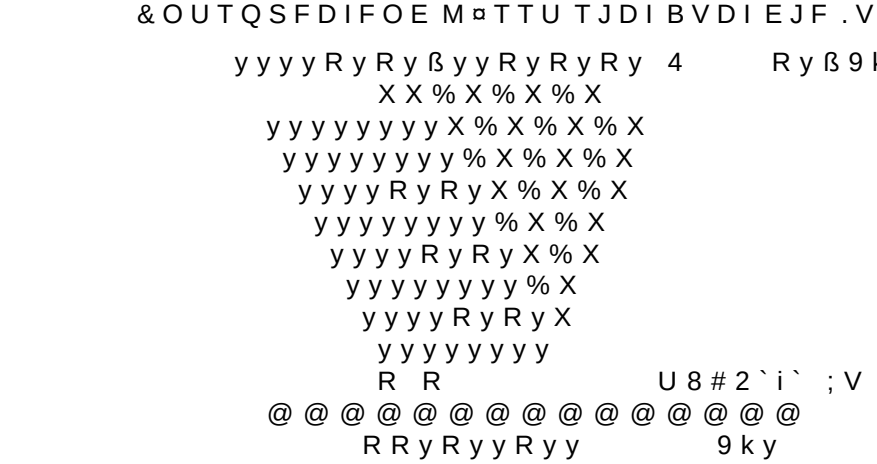
Damit können wir nun die Zahlen 0 bis 255 darstellen, z.B.:

$$\begin{aligned} &00000000 \to 0\,, \\ &00000011 \to 3\,, \\ &01011000 \to 88\,, \\ &11111111 \to 255\,. \end{aligned}$$

%BT "EEJFSFO GVOLUJPOJFSU OVO HFOBVTP XJF JIS FT BVT EFS
EBT ;FIOFSTZTUFN WFSXFOEFU XJSE HFMFSOU IBCU #FJTQJFM



&OUTQSFDIFOE M¤TTU TJDI BVDI EJF .VMUJQMJLBUJPO EVSDI÷¼IS



8FOO JIS HFOBV IJOTFIU EBOO CFNFSLU JIS EBTT XJS[XFJ #ZUFT
BCFS EBT &SHFCOJT CFO¶UJHU FJO #JU NFIS %BT ÷¼ISU[V EFN 1
BVG FJOFN $PNQVUFS EJFTF .VMUJQMJLBUJPO EVSDI÷¼ISU EBT I
MJOLT BCHFTDIOJUUFO XJSE VOE EBT &SHF1640JTTU EBTT EFS $PNQ
%BT HMFJDIF QBTTJFSU CFJ255JEJEHNB#[ZUJFIMXUFOEBJT&$VHFCOJT JTU
0 JO EFS 'BDITeSCFDSMEBWGDIOFU
$PNQVUFS L¶OOFO BVDI OFHBUJWF ;BIMFO EBSTUFMMFO %B[V
XFC255+1[VN &SHFCOUJEBO1EBUT &SH1FCOT BMTP
%JFT OFOOU NBO EBT â;XFJFSLPNQMFNFOUã EFO #FHSJ8N¼TTU J
NFSLFO 8JDIUJH JTU EBTT I¶DITUF #JU[FJHU EBTT FT TJDI VN F
EFMU %BNJU L¶OOFO EBOO NJU FJOFN #ZUF EJF ;BIMFO CJT

8JF XJSE OVO BVT FJOFS QPTJUJWFO ;BIM FJOF /FHBUJWF %BT QE
#JUT JOWFSUJFSU EBT IFJŸU BVT XJSE VOE BVT FJOF VOE

2.5 Lauflicht

Im folgenden C-Programm wird ein sogenanntes “Lauflicht” erzeugt. Dabei wird ein Sternchen von links nach rechts über den Bildschirm bewegt. Das Programm läuft dabei unendlich lange, bis es mit `Strg+C` abgebrochen wird.

```
O/ 2 7 B Mn2n : L l n a P l _ * 1
OB M + H m1 /b2 1 / B Q X ? =
OB M + H m1 /m2 M B b i / X ? =

B M iK  BUMV  &
    r ? B HU2R V  &
        7 Q `UB M iB 4  y cB l  k y cBY Y V  &
            T ` B MüJ7$` ] V c
            7 Q `UB M iD 4  y cD l  k y cDY Y V  &
                B 7 UB  4 4 DV
                    T ` B MüJ7 ] V c
                2 H b 2
                    T ` B MüJ7 ] V c
            '
            m b H 2U2RTy y y y y V c
        '
    '

```

Tastenkombination	Zeichen	Bedeutung
AltGr + 3	#	Hashtag
AltGr + +	~	Tilde

B.1.2 Referenz

Die folgende Liste von Tastenkombination gilt für viele, aber nicht für alle Programme:

Tastenkombination	Bedeutung
Markieren	
Esc	Markierung au f eben
Shi f +Cursor	Text markieren (alternativ mit Maus)
Strg+A	Alles auswählen (“select all”)

B.2 C

Zahl in wissenschaftlicher Notation dargestellt wird. Zum Beispiel bedeutet `1.2E-38` die Zahl $1.2 \cdot 10^{-38}$, also eine sehr kleine Zahl, während `3.4E+38` die Zahl $3.4 \cdot 10^{38}$ ist, also eine sehr große Zahl. Man kann sagen, dass die Zahl nach "E" die Zahl der Stellen angibt, um die das Komma verschoben werden muss, um die Zahl in Standardform zu bringen. Ein negativer Exponent bedeutet, dass das Komma nach links verschoben wird, während ein positiver Exponent bedeutet, dass das Komma nach rechts ver-

Anweisungsblock Ein Anweisun
weisungen, die zusammengefasst und
C++ wird ein Anweisungsblock durch g
Anweisungsblock kann überall dort verw
erwartet wird, z.B. in Kontrollstrukturen wi
weisungsblocks können Variablen deklariert
sichtbar sind (lokale Variablen). Dies ermöglicht
und verhindert Namenskonflikte.

Blöcke können auch verschachtelt werden, d.h. ein
halb eines anderen Anweisungsblocks enthalten sein. Die
den Gültigkeitsbereich von Variablen zu begrenzen und die L
verbessern.

Aufpassen solltet ihr jedoch mit der Benennung von Variablen in
ten Blöcken. Wenn ihr in einem inneren Block eine Variable mit dem gleich
men wie in einem äußeren Block deklariert, wird die äußere Variable innerhalb des
inneren Blocks ”verdeckt”(englisch: *shadowed*). Das bedeutet, dass ihr innerhalb des inneren Blocks nur auf die innere Variable zugreifen könnt, während die äußere Variable vorübergehend nicht zugänglich ist. Dies kann zu Verwirrung führen und sollte vermieden werden, indem ihr eindeutige Namen für Variablen verwendet.Beispiel:

```
B M iK  BUMV  &
    B M it  4  R y cf  › m 8 2 `2  `B  # H 2
    &
        B M it  4  k y cf f  B M M 2 ò 2 ` B  # Hp22 ` / 2 + F/ iB 2 š m 8 2 ` 2
        T ` B MüJ7A M M 2 ĺ ,2  W $M] -x t0&tBfu t  m b ;  #, 2k y
    '
    T ` B MüJ7š m ,8 2 `t2,  W
```

2 H b 2 B 7

Die `else if`-Anweisung wird verwendet, um eine weitere Bedingung zu überprüfen, wenn die vorherige `if`-Bedingung falsch war. Die Syntax lautet:

```
B 7 U" 2 / B M ; m \W M ;&R
    f f * Q / 2 - / 2 ` m b ; 2 7 9 ?r`B ` /- r 2 M M 2 / B M ; m M ; ?R` B b i
'  2 H b 2 B 7 U" 2 / B M ; m \W M ;&k
    f f * Q / 2 - / 2 ` m b ; 2 7 9 ?r`B ` /- r 2 M M 2 / B M ; m M ; ?k` B b i
'
```

2 H b 2

Die `else`-Anweisung wird verwendet, um einen Codeblock auszuführen, wenn keine der vorherigen Bedingungen wahr ist. Die Syntax lautet:

```
B 7 U" 2 / B M ; m \W M ;&R
    f f * Q / 2 - / 2 ` m b ; 2 7 9 ?r`B ` /- r 2 M M 2 / B M ; m M ; ?R` B b i
'  2 H b 2 B 7 U" 2 / B M ; m \W M ;&k
    f f * Q / 2 - / 2 ` m b ; 2 7 9 ?r`B ` /- r 2 M M 2 / B M ; m M ; ?k` B b i
'  2 H b 2 &
    f f * Q / 2 - / 2 ` m b ; 2 7 9 ?r`B ` /- r 2 M M# 2 B M / 2 `
    " 2 / B M ; m M r, 2?M B b i
'
```

Eine Bedingung ist ein Ausdruck, der entweder `true` (wahr) oder `false` (falsch) ergibt. In C und C++ werden Bedingungen durch Vergleiche oder sogenannte logische Ausdrücke gebildet.

Ein Vergleich bildet sich aus zwei Werten und einem Vergleichsoperator. Wenn wir zum Beispiel feststellen wollen ob zwei Zahlen `a` und `b` gleich sind, dann schreiben wir `a == b`. Dabei wird == als Operator bezeichnet (um genau zu sein, als Vergleichsoperator).

Liste aller Vergleichsoperatoren: Mögliche Vergleiche sind:

4 4
: Gleichheit: Überprüf, ob zwei Werte gleich sind.

5 4
: Ungleichheit: Überprüf, ob zwei Werte ungleich sind.

I
: Kleiner als: Überprüf, ob der linke Wert kleiner als der rechte Wert ist.

=
: Größer als: Überprüf, ob der linke Wert größer als der rechte Wert ist.

I 4
: Kleiner oder gleich: Überprüf, ob der linke Wert kleiner oder gleich dem rechten Wert ist.

= 4
: Größer oder gleich: Überprüf, ob der linke Wert größer oder gleich dem rechten Wert ist.

Logische Operatoren werden verwendet, um mehrere Bedingungen zu kombinieren oder zu negieren. Die wichtigsten logischen Operatoren in C und C++ sind:

Logisches UND: Gibt `true` zurück, wenn beide Bedingungen wahr sind.

% %
: Logisches ODER: Gibt `true` zurück, wenn mindestens eine der Bedingungen wahr ist.

5
: Logisches NICHT: Negiert die Bedingung, d.h. gibt `true` zurück, wenn die Bedingung falsch ist, und `false` zurück, wenn die Bedingung wahr ist.

Schleifen C und C++ bieten drei Arten von Schleifen an: `for`, `while` und `do-while`. Jede Schleife hat ihre eigenen Anwendungsfälle und Syntax.

7 QSchleife
: Die `for`-Schleife wird verwendet, wenn die Anzahl der Schritte im Voraus bekannt ist. Sie hat die folgende Syntax:

```
&
    A M B i B   H B b Bc2 ` m M ;
    r ? B HU2 2 / B M ; mV M&
        f f  * Q / 2  / 2 ` r B 2 / 2 ` ? Q Hmi b ; 2 7 9 ?r`B ` /
        A M F ` 2 Kc2 M i
    '
'
```

Die Initialisierung wird nur einmal zu Beginn der Schleife ausgeführt. Sie dient gewöhnlich dazu, eine Zählvariable zu deklarieren und zu initialisieren. Die Bedingung wird vor jedem Durchlauf überprüft, und das Inkrement wird am Ende jedes Durchlaufs ausgeführt (Inkrement heißt wörtlich "Erhöhung"). Wenn wir zum Beispiel 1 bis 10 zählen wollen, dann schreiben wir:

```
7 QÙB M iB 4  R cB I 4  R y BY Y V  &
    T ` B MÙ]7W $M] -  BV c
'
```

oder von 100 in 10-er Schritten bis 0 zählen wollen, dann schreiben wir:

```
7 QÙB M iB 4  R y y B = 4  y cB @ 4  R y V  &
    T ` B MÙ]7W $M] -  BV c
'
```

Funktionen Um ein Programm in kleinere, wiederverwendbare Teile zu zerlegen, können wir in C und C++ Funktionen verwenden. Eine Funktion ist ein benannter Codeblock, der eine bestimmte Aufgabe ausführt und optional Werte zurückgeben kann. Funktionen ermöglichen es uns, den Code zu organisieren, die Lesbarkeit zu verbessern und die Wiederverwendbarkeit zu erhöhen.

Eine Funktion besteht aus einem Funktionskopf, der den Namen der Funktion, die Parameter und den Rückgabewert definiert, und einem Funktionskörper, der die Anweisungen enthält, die ausgeführt werden, wenn die Funktion aufgerufen wird. Die Syntax für die Definition einer Funktion lautet:

```
_ 9 + F ;  # 2 r 2 6 `m M F i B Q MÙ]SM ` KK2 2 i 2 ` H BV b &2
    f f   M r 2 B b m M ; 2 M
'
```

Der Rückgabewert gibt den Datentyp des Wertes an, den die Funktion zurückgibt (siehe oben B.2.1). Wenn die Funktion keinen Wert zurückgibt, wird der Rückgabewert als `void` angegeben. Die Parameterliste enthält die Eingabewerte, die an die Funktion übergeben werden,ertyp des Wertes an, de] Š w w ž die w "H w]